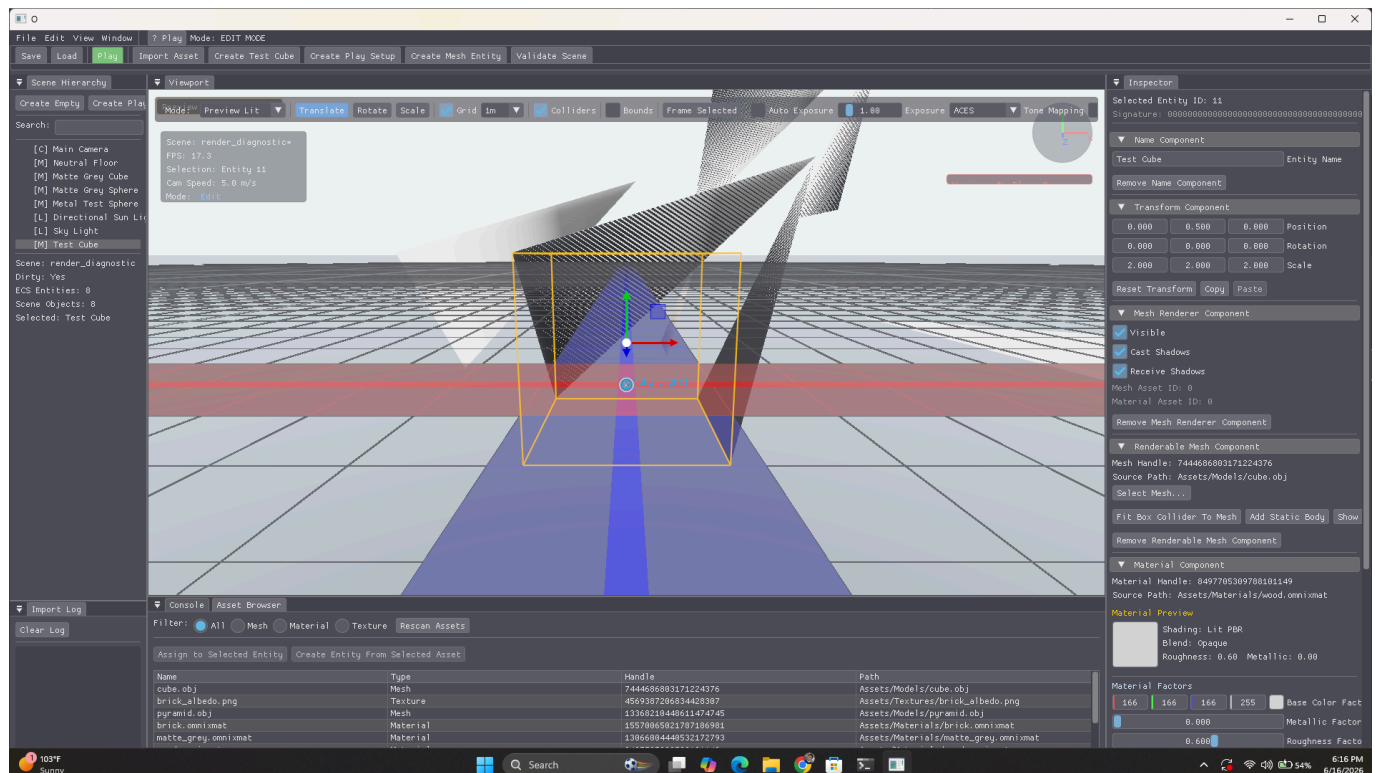


Mesh Volume Rendering Diagnosis



Summary

The viewport currently shows editor bounds, gizmos, grid overlays, and debug volumes, but actual mesh surface volume is not rendering reliably. This is not primarily a lighting or tonemapping issue. The strongest evidence points to a backend architecture mismatch in the GPU-driven indirect rendering path.

The renderer is issuing indirect draws as if all meshes live in one shared vertex/index buffer arena. The current **Mesh** implementation uploads every mesh into its own separate vertex and index buffers. Because of that mismatch, indirect draw commands cannot correctly address heterogeneous scene meshes.

Primary Failure

The active/default visibility path is GPU-driven indirect rendering:

- `Renderer::VisibilityMode::GPUFrustumIndirect`
- `Renderer::VisibilityMode::GPUFrustumOcclusion`

In the GBuffer pass, the renderer binds only the first mesh in the render queue, then executes indirect draw commands for all visible instances.

Relevant path:

- `Rendering/Core/Renderer.cpp`
- `GBufferPass`
- indirect branch around `vkCmdDrawIndexedIndirect`

The problematic behavior is:

```

const auto& items = renderQueue.getItems();
if (!items.empty() && items[0].mesh != nullptr) {
    items[0].mesh->bind(cmd);

    if (items[0].material != nullptr) {
        items[0].material->bindDescriptorSet(cmd, m_GBufferIndirectPipelineLayout,
1);
    }

    VkBuffer indirectCmdBuf =
m_IndirectCommandBuildPass.GetIndirectCommandBuffer(frameIndex);
    if (indirectCmdBuf != VK_NULL_HANDLE && m_GpuIndirectDrawCount > 0) {
        vkCmdDrawIndexedIndirect(cmd, indirectCmdBuf, 0, m_GpuIndirectDrawCount,
sizeof(VkDrawIndexedIndirectCommand));
    }
}

```

This only works if all indirect commands reference subranges inside the same currently bound vertex and index buffers. That is not true in the current backend.

Current Mesh Buffer Model

Each `Mesh` owns independent GPU buffers.

Relevant path:

- `RenderingEngine/Renderer/scene/Mesh.cpp`
- `RenderingEngine/Renderer/scene/Mesh.h`

`Mesh::init()` creates:

- one vertex buffer per mesh
- one index buffer per mesh
- `firstIndex = 0`
- `vertexOffset = 0`

That means per-mesh offsets in indirect commands are not meaningful across multiple mesh objects. Binding `items[0].mesh` and then drawing indirect commands for other meshes will still read from `items[0]`'s buffers.

Expected result:

- missing mesh volume
- wrong mesh geometry
- corrupted triangles
- oversized/invalid shapes
- shadow/depth artifacts
- object IDs or bounds appearing while shaded surfaces are absent

Indirect Command Builder Mismatch

The compute shader builds `VkDrawIndexedIndirectCommand`-like records using `mesh.indexCount`, `mesh.firstIndex`, and `mesh.vertexOffset`.

Relevant paths:

- `Rendering/Visibility/IndirectCommandBuildPass.cpp`
- `shaders/build_indirect_commands.comp`
- `Rendering/GPUScene/GPUMeshDrawData.h`

The shader writes:

```
commands[visibleIndex].indexCount = mesh.indexCount;
commands[visibleIndex].instanceCount = 1;
commands[visibleIndex].firstIndex = mesh.firstIndex;
commands[visibleIndex].vertexOffset = mesh.vertexOffset;
commands[visibleIndex].firstInstance = visibleIndex;
```

This is correct only for a renderer that has a shared geometry buffer. The current renderer does not.

Material Binding Failure

The indirect GBuffer path also binds only the first material descriptor set:

```
if (items[0].material != nullptr) {
    items[0].material->bindDescriptorSet(cmd, m_GBufferIndirectPipelineLayout, 1);
}
```

That means all indirect draws sample textures from the first queued material descriptor set. Even if the shader receives a material index through GPU scene buffers, the actual sampled texture descriptors are still not per-draw or bindless.

Expected result:

- incorrect material appearance
- fallback-looking materials
- wrong albedo/normal/roughness sampling
- flat or washed-out surfaces
- invisible or misleading material output

Depth And Shadow Artifacts

The same architectural issue affects depth and shadow-related passes.

The depth prepass indirect branch also binds the first mesh and then executes indirect draw commands. If indirect commands represent multiple meshes, the depth buffer becomes unreliable.

Consequences:

- invalid depth buffer
- bad GBuffer background/foreground classification
- broken SSAO input
- broken shadow projection/sampling
- large black or striped artifacts
- geometry-like shapes appearing where no mesh should be

The screenshot's large black triangular/striped shapes are consistent with this class of failure.

Why Bounds And Gizmos Still Appear

Editor overlays are not proof that mesh volume rendered.

The screenshot shows:

- selected entity bounds
- transform gizmo
- grid overlay
- light/debug volumes
- labels

These are drawn through editor/debug paths and can appear even when the actual GBuffer mesh surface path is broken. This explains why the cube can be selected and outlined while its shaded mesh volume is absent or visually corrupted.

Lighting Caveat

The screenshot is in **Preview Lit** mode and shows **Preview Lighting Active**.

That means authored scene lights are not the lighting source of truth for the screenshot. This should be corrected when judging final lighting. However, preview lighting does not explain missing mesh volume by itself. It only makes lighting/material evaluation unreliable.

Diagnostic Isolation Plan

Use the following order to isolate the failure:

1. Switch visibility mode to **CPU Driven**.
2. Switch viewport mode to **Lit** or **AlbedoOnly**, not **Preview Lit**.
3. Disable shadows.
4. Disable grid, collider, bounds, and debug overlays.
5. Verify whether cube and floor mesh surfaces appear.
6. If CPU path works, switch to **GPU Frustum Only**.
7. If GPU-frustum CPU draws work, switch to **GPU Frustum + Indirect**.
8. Confirm the break appears only at the indirect path.

Expected result:

- CPU path should render mesh volume if assets/materials are valid.

- Indirect path is expected to fail until geometry buffers are unified or indirect drawing is grouped by bound mesh.

Short-Term Fix

Disable GPU indirect modes by default.

Recommended temporary default:

```
m_VisibilityMode = VisibilityMode::CPUDriven;
```

or:

```
m_VisibilityMode = VisibilityMode::GPUFrustumOnly;
```

GPUFrustumOnly can still use GPU visibility results while drawing with CPU-side per-item mesh binding. That matches the current separate-buffer mesh architecture better than full indirect rendering.

Also hide or clearly gate:

- GPU Frustum + Indirect
- GPU Frustum + Occlusion

until the backend supports them correctly.

Long-Term Fix

Implement a real GPU-driven geometry backend.

Required pieces:

- Global vertex buffer arena.
- Global index buffer arena.
- Per-mesh **firstIndex** and **vertexOffset** into those global buffers.
- Stable mesh table uploaded to GPU.
- Stable material table uploaded to GPU.
- Bindless texture descriptors or texture arrays.
- Per-draw material/texture indexing in shader.
- Indirect command generation grouped or indexed correctly.
- Robust debug views for:
 - visible instance IDs
 - mesh index
 - material index
 - indirect draw count
 - object ID output
 - GBuffer albedo/normal/depth

Alternative long-term path:

- Keep separate mesh buffers.
- Batch indirect draws per mesh/material group.
- Bind the correct mesh buffers before each indirect batch.

This is less scalable than a shared geometry arena but much closer to the current renderer architecture.

Acceptance Criteria For Fix

The renderer should satisfy:

- CPU-driven mode renders mesh volume correctly.
- **GPUFrustumOnly** renders the same geometry as CPU-driven mode.
- Indirect mode is disabled or renders the same geometry as CPU-driven mode.
- **AlbedoOnly** clearly shows object surfaces.
- **Depth** mode shows stable, coherent depth silhouettes.
- **Object ID** mode shows selected object IDs only where mesh surfaces exist.
- No large black/striped geometry artifacts.
- Validation layers remain clean.

Conclusion

The missing mesh volume is most likely caused by enabling a GPU indirect draw path before the engine has the required shared geometry-buffer backend. Current meshes are stored in independent vertex/index buffers, while indirect drawing assumes all visible instances can be drawn from a single bound mesh buffer.

Until this mismatch is fixed, the engine should default to CPU-driven or GPU-frustum CPU drawing and treat full indirect rendering as experimental/disabled.